ASIAN SCHOOL OF BUSINESS

Programme	**:**	**BCA (Honours) AI and Data Science**
Course Name	**:**	**Database Management Systems**
Course Code	**:**	25-9222-304
Given Date	**:**	1-10-26
Date of Submission	**:**	12-10-26

Assignment - 4

Topic: Orders and Order Details Table

Submitted to,

Athira A G

Assistant Professor

Submitted by,

Name: Shivganga R T

Class: BCA

SKILL DEVELOPMENT WEEK 4

ORDERS AND ORDER DETAILS MANAGEMENT SYSTEM

INTRODUCTION

The **Orders** and **Order Details Management System** is a database project designed to capture and track customer transactional history using **MySQL**. Its objectives are to create highly minimal **Orders** and **Order_Details** tables, manage customer product transactions, store core details such as order dates, item quantities, and total money amounts, perform entry insertion and modification actions, and generate complete customer order history reports.

2. DATABASE DESIGN

2.1 Orders Table

Stores general invoice parameters for a transaction.

Attribute	Data Type	Description
order_id	INT	Primary key
customer_name	VARCHAR(100)	Name of the buyer
order_date	DATE	Date the transaction took place
total_amount	DECIMAL(10,2)	Total financial value of the order

2.2 Order_Details Table

Stores structural breaking points for individual items inside an order.

Attribute	Data Type	Description
detail_id	INT	Primary key
order_id	INT	Foreign key referencing Orders
product_name	VARCHAR(150)	Name of the purchased item
quantity	INT	Number of items bought
unit_price	DECIMAL(10,2)	Price per single item

3. SQL Implementation

3.1 Create the Database

```
CREATE DATABASE OrderTrackingDB;
USE OrderTrackingDB;
```

3.2 Create the Tables

- Create **Orders Table**

```
CREATE TABLE Orders (order_id INT PRIMARY KEY, customer_name VARCHAR(100), order_date DATE,total_amount DECIMAL(10,2));
```

- Create **Order_Details Table**

```
CREATE TABLE Order_Details (detail_id INT PRIMARY KEY, order_id INT, product_name VARCHAR(150), quantity INT, unit_price DECIMAL(10,2),
FOREIGN KEY (order_id) REFERENCES Orders(order_id));
```

4. Insert Sample Records

- Insert data into **Orders Table**

```
INSERT INTO Orders VALUES (1001, 'Alice Smith', '2026-10-10', 1045.00), (1002, 'Bob Jones', '2026-10-11', 120.00), (1003, 'Alice Smith', '2026-10-12', 50.00);
```

- Insert data into **Order_Details Table**

```
INSERT INTO Order_Details VALUES (1, 1001, 'Wireless Laptop', 1, 999.00),(2, 1001, 'Wireless Mouse', 2, 23.00),(3, 1002, 'Running Shoes', 1, 120.00), (4, 1003, 'Water Bottle', 2, 25.00);
```

5. Order Insertion and Modification Operations

5.1 Perform Order Insertion

Adds a new independent transaction to the log when a purchase takes place.

- Insert a new order entry

  INSERT INTO Orders VALUES (1004, 'Charlie Brown', '2026-10-13', 80.00);

- Insert the matching breakdown details for the order

  INSERT INTO Order_Details VALUES (5, 1004, 'Mechanical Keyboard', 1, 80.00);

5.2 Perform Order Modification

Modifies existing metrics inside the system safely using target updates.

- Update the quantity and price adjustments for a specific detail line

  UPDATE Order_Details SET quantity = 3 WHERE detail_id = 2;

- Update the parent tracking total amount to match updated calculation values

  UPDATE Orders SET total_amount = 1068.00 WHERE order_id = 1001;

6. Customer Order History Reports

6.1 Complete Customer Transaction Log Report

Combines structural fields across both entities to display full itemized history metrics.

SELECT o.order_id, o.customer_name, o.order_date, d.product_name, d.quantity, d.unit_price, o.total_amount FROM Orders o JOIN Order_Details d ON o.order_id = d.order_id ORDER BY o.order_date DESC, o.order_id;

6.2 Customer Financial Spending Summary Report

Generates high-level business analytics tracking order totals and total cumulative capital generated per buyer.

```
SELECT customer_name, COUNT(order_id) AS total_orders_placed,
SUM(total_amount) AS total_money_spent FROM Orders GROUP BY
customer_name ORDER BY total_money_spent DESC;
```

CONCLUSION

The **Orders** and **Order Details Management System** establishes a highly effective framework for processing e-commerce operations in **MySQL**. By using only required attributes and completely discarding conditional query logic or automated id structures, the schema yields rapid lookups while maintaining clean referential integrity. Implementing clear insertion patterns, selective modifier fields, and aggregated spending profiles delivers clean analytical utility for monitoring sales pipelines and buyer purchasing patterns over time.